SHRI VAISHNAV VIDYAPEETH VISHWAVIDYALAYA, INDORE

SHRI VAISHNAV INSTITUTE OF INFORMATION TECHNOLOGY

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



Project Synopsis

of project titled

VoteChain: Blockchain Based Voting System

submitted in the subject of

Blockchain [BTDSC718N]

Submitted by: UMANG GOSWAMI [22100BTCSAII11062]

ABSTRACT

Conventional electronic voting systems store ballots in a central database that a single administrator can silently modify, making election results difficult to trust and audit. To address this issue, this work proposes VoteChain, a web-based voting system that records every accepted ballot on a blockchain implemented from scratch in Python.

Voters register and sign in using bcrypt-protected credentials, while an administrator creates elections, manages candidates, and enrols eligible voters. When a ballot is cast, the server enforces voter eligibility and the one-vote rule within a single SQLite transaction, supported by database constraints. Each ballot is stored anonymously, containing only the selected candidate, a random receipt, and a keyed voter tag rather than the voter's identity.

Pending ballots are sealed into blocks using SHA-256 hashing and a configurable Proof of Work mechanism, with each block linked to its predecessor. A blockchain explorer verifies the integrity of the entire chain on demand and allows voters to confirm their ballots using their receipts. Election results are calculated exclusively from validated ballots contained in verified blocks, ensuring that altered blocks are detected and excluded from the results.

The system is developed using Python, Streamlit, and SQLite. It preserves data across restarts and can be deployed on Streamlit Community Cloud or any Python-compatible hosting platform. Configurable parameters, including mining difficulty and block size, allow the system to be adapted to different operational environments.

This work demonstrates how hash-linked records, strict server-side validation, and public verifiability can support tamper-evident and transparent elections. VoteChain provides a simple, low-cost voting platform for institutions seeking improved election integrity and auditability.

1. INTRODUCTION

Electronic voting replaces paper ballots with digital ones to make elections faster, cheaper and more accessible. Its central challenge is trust: voters must be sure that only eligible people vote, that each person votes once, that ballots stay secret and that the final count matches what was cast. Most systems rely on a central database, so the outcome depends on the honesty and security of whoever controls it.

A blockchain is an append-only list of blocks in which each block stores the cryptographic hash of its predecessor. Changing an old record changes its hash and breaks every later link, so tampering becomes detectable. The idea was popularised by Bitcoin[1], whose Proof of Work builds on the hashcash scheme[2] by requiring a block hash to meet a difficulty target. Hashing follows the SHA-256 standard[3].

Researchers have applied these properties to voting. Hjálmarsson et al. proposed a blockchain-based election system[4], Kshetri and Voas discussed the promise and risks of blockchain-enabled voting[5], and Hardwick et al. described a protocol combining decentralisation with voter privacy[6]. Common themes are transparency, integrity and auditability, balanced against privacy and usability.

This project follows that line of work. It builds a complete, working election application, covering accounts, election management, vote casting, mining, verification and results, on a custom blockchain, so that every part of the design can be studied, run and extended. Such a design also makes the underlying concepts easy to demonstrate in a classroom setting.

2. PROBLEM DOMAIN

Small-scale online elections, such as those held by colleges, clubs and societies, usually depend on spreadsheets, form tools or a single database. The exact problem is that the result cannot be independently verified. An administrator or intruder can add, remove or change votes without leaving a trace. The same person may vote more than once when rules are enforced only in the interface. Linking votes to named voters also threatens secrecy.

The objectives of the proposed work are:

- To provide secure voter registration and login using hashed passwords.
- To let an administrator create elections, manage candidates and control the voter roll.

- To guarantee one vote per eligible voter using server-side checks and database constraints.
- To implement a blockchain from scratch with Proof of Work and hash-linked blocks.
- To separate voter identity from ballot data.
- To verify chain integrity and compute results only from validated ballots.
- To keep all data persistent across application restarts.

3. SOLUTION DOMAIN

Requirements. Functional needs are registration, login, election and candidate management, voting, mining, a blockchain explorer and results. Non-functional needs are integrity, privacy, persistence and simplicity.

Architecture. The system is layered into Python modules for blockchain logic, database access, authentication, voting and the Streamlit interface. The interface only calls services and holds no rules. Session state stores just a user id, and role, eligibility and voting status are re-read from the database on every page run.

Blockchain design. A block contains an index, timestamp, transactions, previous hash, nonce and current hash, together with the difficulty it was mined at. The hash is SHA-256 over the canonical JSON of these fields. Mining increments the nonce until the hash begins with the required number of zeros (three by default). Verification checks the genesis block, indices, hash links, recomputed hashes and the Proof of Work target.

Voting algorithm. In one write transaction the server checks that the election is open, the voter is on the roll, has not voted, and that the candidate belongs to the election. It then records a participation row and an anonymous pending ballot holding the election, candidate, a random receipt ID and a voter tag, computed as HMAC-SHA256 of the election and user using a server secret[7]. Pending ballots are then mined into blocks.

Schema. Seven tables are used: users, elections, candidates, eligible_voters, voter_participation, pending_ballots and blocks. Primary keys, foreign keys and triggers enforce one vote per voter, voting only while an election is open, and append-only blocks.

Election lifecycle. An election moves from draft to open to closed. Candidates and the voter roll are frozen once it opens, and closing it mines any remaining pending ballots so that the result is final.

Results. Votes are counted only from verified blocks, after every transaction is re-validated for candidate, eligibility and duplicates. An audit view compares participation records with accepted ballots to expose inconsistencies.

4. SYSTEM DOMAIN

Software:

- Python 3 is used for rapid development and its standard library (hashlib, hmac, sqlite3), which suffices to build the blockchain from scratch.
- Streamlit builds the interface entirely in Python[9].
- SQLite provides a zero-configuration, transactional database with constraint and trigger support[10].
- The bcrypt library gives adaptive, salted password hashing[8].
- Pandas supports tables and charts.
- Git and GitHub provide version control, and the app can be deployed on Streamlit Community Cloud or any Python host.

Hardware: A computer with a dual-core processor, 4 GB RAM and 1 GB of free storage is sufficient, as low mining difficulty keeps CPU load small. No graphics processor or special hardware is needed, and a modern web browser is the only client requirement.

Environment: Windows, Linux or macOS with a Python virtual environment; settings are supplied through environment variables or Streamlit secrets, and a terminal with any text editor or IDE is enough for development.

5. APPLICATION DOMAIN

VoteChain is suitable for student councils, clubs, societies, housing associations, cooperatives, and departmental or corporate elections that require a low-cost, auditable voting platform. Potential extensions include support for multiple elections running in parallel, multi-node deployments in which several parties maintain copies of the blockchain, increased mining difficulty, and enhanced privacy techniques such as blind signatures or zero-knowledge proofs.

For voters, the process requires only a few clicks, and each voter receives a receipt to verify their ballot. The blockchain can be inspected and independently verified, while administrators can demonstrate that election results are derived exclusively from validated ballots. The application can be launched with a single command, enabling non-specialist administrators to set up and operate the system with minimal technical expertise.

6. EXPECTED OUTCOME

i. A working Streamlit application with registration, login and administrator controls.
ii. One vote per eligible voter, enforced on the server and in the database.
iii. A custom blockchain with Proof of Work and hash-linked blocks.
iv. An explorer that detects tampering and verifies ballot receipts.
v. Results computed from validated ballots only.
vi. Persistent data, documented and deployment-ready code.
vii. Clear setup instructions so the project can be installed and run on any computer.

7. REFERENCES

[1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008.

[2] A. Back, “Hashcash—A denial of service counter-measure,” technical report, Aug. 2002.

[3] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS PUB 180-4, Aug. 2015.

[4] F. Þ. Hjálmarsson, G. K. Hreiðarsson, M. Hamdaqa, and G. Hjálmtýsson, “Blockchain-based e-voting system,” in Proc. IEEE 11th Int. Conf. Cloud Computing (CLOUD), 2018, pp. 983–986.

[5] N. Kshetri and J. Voas, “Blockchain-enabled e-voting,” IEEE Software, vol. 35, no. 4, pp. 95–99, Jul./Aug. 2018.

[6] F. S. Hardwick, A. Gioulis, R. N. Akram, and K. Markantonakis, “E-voting with blockchain: An e-voting protocol with decentralisation and voter privacy,” in Proc. IEEE Int. Conf. Internet of Things (iThings), Green Computing and Communications (GreenCom), Cyber, Physical and Social Computing (CPSCom), and Smart Data (SmartData), 2018, pp. 1561–1567.

[7] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-hashing for message authentication,” RFC 2104, Internet Engineering Task Force (IETF), Feb. 1997.

[8] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annual Technical Conf., 1999, pp. 81–91.

[9] Streamlit Inc., “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io/

[10] SQLite Consortium, “SQLite documentation.” [Online]. Available: https://www.sqlite.org/docs.html